实验六 基于 PID 控制器的 ROS 乌龟点镇定

一、实验目标

1. 理解 PID 控制器三大环节的工作原理，明晰位置式 PID、增量式 PID 的数学公式、优缺点与适用场景，掌握 PID 参数整定的基本流程与调参规律。

2. 掌握 ROS 话题通信原理，能够通过订阅乌龟位姿、发布速度指令，搭建完整的机器人闭环控制系统。

3. 熟练掌握纯 PID 算法、ROS 乌龟点镇定、ROS 乌龟速度控制代码的编写、调试与运行，熟练编写 Launch 文件实现多节点统一启动。

二、实验环境

操作系统：Ubuntu 20.04 LTS

ROS 版本：ROS Noetic

编程语言：Python 3.8+

三、实验内容及步骤

本次实验以 turtlesim 仿真乌龟为对象，实现二维平面定点 PID 闭环精准控制。实验依托 ROS 双向通信机制，订阅/turtle1/pose 话题采集乌龟坐标与姿态数据，通过/turtle1/cmd_vel 话题发布速度指令调控乌龟运动。系统基于负反馈 PID 算法修正位置与角度误差，配合速度限幅优化，使乌龟稳定精准抵达目标点位。

3.1 PID 基本原理

PID（比例-积分-微分）是工业领域应用最广泛的负反馈控制器，不断计算系统期望值与实际值的误差，根据误差实时调整输出量，让被控对象逼近目标状态。

连续域 PID 公式：$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$

式中：e(t) 为系统误差（目标值−实际值）；K_p 比例系数、K_i积分系数、K_d 微分系数。

比例环节（P）：对当前误差即时响应。K_p 越大，系统响应速度越快；参数过大会引发剧烈振荡，降低稳定性。

积分环节（I）：累积历史误差，消除系统静态误差。K_i 越大，消差速度越快；易增大超调、降低响应速度。

微分环节（D）：预判误差变化趋势，提前修正输出。可抑制振荡、减小超调；对传感器噪声敏感。

计算机无法处理连续信号， PID 离散化分为位置式 PID 和增量式 PID。

3.1.1 增量式 PID

增量式 PID 算法通过计算相邻两个采用周期控制器输出的增量$\Delta u(\mathrm{k})$来实现控制，其表达式可由位置式 PID 推导得出：

$\Delta u(\mathrm{k}) = u(\mathrm{k}) - u(\mathrm{k}-1)$

$=K_p[e(k)-e(k-1)]+k_iTe(k)+k_d(e(k)-2e(k-1)+e(k-2))$ 。

执行机构的位置由前一时刻的输出值$u(\mathrm{k}-1)$加上增量$\Delta u(\mathrm{k})$得到， 即:

$u(k)=u(k-1)+\Delta u(k)$

增量式 PID 的优点：无需积累历史误差，只需保存前 1 到 2 个周期的误差值，计算量小，可有效避免积分饱和问题；输出增量较小，执行机构动作平缓，适合对控制量变化速率敏感的系统（如电机调速系统中的转速调节）。

增量式 PID 的缺点：依赖前一时刻的输出值，若计算机出现故障导致输出值异常，可能影响后续控制；物理意义不如位置式直观，需要通过增量的累加才能得到执行机构的绝对位置。

3.1.2 位置式 PID

位置式输出控制量$u(k)=K_pe(k)+K_i\sum_{j=0}^{k}e(j)+K_d(e(k)-e(k-1))$，该算法适用点位、位置精准控制。

位置式 PID 的优点：物理意义明确，输出直接对应执行机构的实际位置，适用于需要精确控制执行机构绝对位置的系统。

位置式 PID 的缺点：每次计算需要积累所有历史误差，计算量大，且容易引起积分饱和（当系统长期处于误差状态时，积分项过大导致控制器输出饱和，造成系统超调或振荡）；对执行机构的冲击较大，若控制器输出值突变，可能导致执行机构动作剧烈。

3.2 工作环境搭建

3.2.1 创建工作空间

```
mkdir -p ~/pid_ws/src
cd ~/pid_ws
```

3.2.2 编译并打开 Vscode

```
catkin_make
code .
```

3.2.3 创建依赖完整的功能包

```
pid_pkg rospy geometry_msgs turtlesim
```

3.2.4 功能包内创建目录

scripts：存放所有 Python 代码

launch：存放 ROS 启动文件

3.3 位置式 PID 乌龟点镇定

控制小乌龟从默认起点运动到指定目标坐标(8.0,8.0)，到达后稳定停止。

3.3.1 库导入

```
#!/usr/bin/env python3
import rospy
from geometry_msgs.msg import Twist
```

from turtlesim.msg import Pose

import math

◆#!/usr/bin/env python3，Linux 系统脚本声明，告诉终端用 python3 解释器运行该文件；

◆import rospy，ROS 核心库，提供节点初始化、发布、订阅、延时、日志打印全部基础 API。

◆from geometry_msgs.msg import Twist，速度消息类型：Twist 包含线速度 linear、角速度 angular，乌龟运动控制唯一输出话题 /turtle1/cmd_vel 标准消息。

◆from turtlesim.msg import Pose，乌龟位姿反馈消息：包含 x/y 坐标、theta 航向角、线速度角速度，订阅 /turtle1/pose 获取实时位置。

◆import math，数学库：用于距离开方、反正切 atan2 角度计算、角度误差归一化 sin/cos 运算。

3.3.2 全局参数定义

```
# PID 参数
Kp = 0.8
Ki = 0.01
Kd = 0.2
# 乌龟初始位姿（turtlesim 默认起点）
current_x = 5.544445
current_y = 5.544445
current_theta = 0.0
# 自定义目标点
target_x = 8.0
target_y = 8.0
# PID 积分、历史误差缓存
dis_err_sum = 0.0
dis_last_err = 0.0
ang_err_sum = 0.0
ang_last_err = 0.0
# 到达阈值
reach_thresh = 0.15
```

◆PID 三参数 Kp/Ki/Kd

1. 比例 Kp：误差越大输出越大，快速缩小偏差；过大震荡、过小响应慢；
2. 积分 Ki：消除稳态静差（乌龟到点停不住、有残余距离就是积分不足）；过大积分饱和；
3. 微分 Kd：抑制震荡、预判误差变化趋势，平滑运动。

◆当前位姿全局变量

1. 初始值是 turtlesim 窗口中心坐标；

2. 设为 global 全局变量：回调函数实时更新，主循环随时读取最新乌龟位置。

◆目标点位 target_x/target_y，修改数值，观察定点跟随效果；窗口范围 0~11，超出会撞墙。

◆PID 缓存变量

1. xxx_error_sum：积分项累加和，每个控制量独立积分，不能共用；

2. xxx_last_error：上一周期误差，用于微分项计算误差变化率。

3.3.3 位姿订阅回调函数

```
def pose_callback(pose_msg):
    global current_x, current_y, current_theta
    current_x = pose_msg.x
    current_y = pose_msg.y
    current_theta = pose_msg.theta
```

◆回调函数触发逻辑：只要 /turtle1/pose 发布新位姿数据，ROS 自动调用该函数，属于异步订阅；

◆global 关键字：函数内部修改外部全局变量，不加 global 只会创建局部变量，主循环坐标永远不变，乌龟完全不动；

◆数据提取：pose_msg.x 水平坐标、pose_msg.y 竖直坐标、pose_msg.theta 航向角。

3.3.4 通用位置式 PID 函数

```
def position_pid(setpoint, current, error_sum, last_error):
    error = setpoint - current
    p_term = Kp * error
    error_sum += error
    i_term = Ki * error_sum
    d_term = Kd * (error - last_error)
    output = p_term + i_term + d_term
    return output, error_sum, error
```

3.3.5 主程序

1.ROS 节点初始化

```
    if __name__ == '__main__':
        rospy.init_node('turtle_position_controller')
         # 2. 创建速度发布器
         pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)
         # 3. 订阅位姿话题，绑定回调函数
        rospy.Subscriber('/turtle1/pose', Pose, pose_callback)
```

```
# 4. 设置循环频率 20Hz
rate = rospy.Rate(20)
```

◆rospy.init_node('节点名')，初始化 ROS 节点，同一工程节点名不能重复；

◆发布端 Publisher，持续向外发送速度指令控制乌龟；

◆订阅端 Subscriber 订阅乌龟实时位姿，收到数据自动执行回调；发布订阅异步并行；

◆rospy.Rate(20)，控制主循环 20Hz 刷新，每轮间隔 0.05s，固定 PID 控制周期，保证微分、积分计算稳定；频率过高 CPU 占用大，过低控制滞后。

2.ROS 主循环

```
while not rospy.is_shutdown():
    # 计算两点直线距离
    distance = math.sqrt((target_x - current_x)**2 + (target_y - current_y)**2)
    # 计算指向目标点的期望角度
    desired_theta = math.atan2(target_y - current_y, target_x - current_x)
    # 角度误差归一化到 [-π, π]
    theta_error = math.atan2(math.sin(desired_theta - current_theta),
                             math.cos(desired_theta - current_theta))
```

◆欧氏距离公式：计算乌龟到目标点直线距离，作为位置 PID 的反馈量；

◆atan2(dy, dx)：求目标相对乌龟的期望朝向角；朝向角直接相减角度误差可能超出 ±180°（如 350° 与 10° 差值 340°，实际只需要转 -20°）；利用 sin/cos 映射把误差强制约束在 [-π, π]，防止乌龟绕大圈反向旋转。

3. 判断是否达到目标点

```
if dist < reach_thresh:
    vel = Twist()
    pub.publish(vel)
    rospy.loginfo("已到达目标点，停止运动！")
    dis_err_sum = 0
    ang_err_sum = 0
    rate.sleep()
    continue
```

4. 调用 PID

```
linear_v, dis_err_sum, now_dis_err = pos_pid(0, dist, dis_err_sum, dis_last_err)
angular_v, ang_err_sum, now_ang_err = pos_pid(0, ang_err, ang_err_sum, ang_last_err)
dis_last_err = now_dis_err
ang_last_err = now_ang_err
```

◆线速度 PID：

■ 设定值 setpoint=0，反馈值是当前距离 distance；

■ 逻辑：距离越大误差越大，PID 输出更大前进速度；距离趋近 0，速度归零；

◆旋转角速度 PID：

◆设定值 setpoint=0，反馈值是角度误差 theta_error；

◆逻辑：朝向偏差大，输出大角速度旋转对齐目标；角度对齐后角速度为 0。

4. 速度限幅

```
linear_velocity = max(-2.0, min(linear_velocity, 2.0))
angular_velocity = max(-2.0, min(angular_velocity, 2.0))
```

◆PID 输出值无边界，参数调大后速度会无穷大，乌龟瞬间冲出窗口；

◆通过 min/max 把线速度、角速度限制在 [-2,2] 区间，平滑、安全控制，工业控制中称为输出限幅。

5. 设置消息并发布

```
vel_msg = Twist()
vel_msg.linear.x = linear_velocity
vel_msg.angular.z = angular_velocity
pub.publish(vel_msg)
```

实例化空 Twist 消息，只赋值前进速度 linear.x、旋转角速度 angular.z；

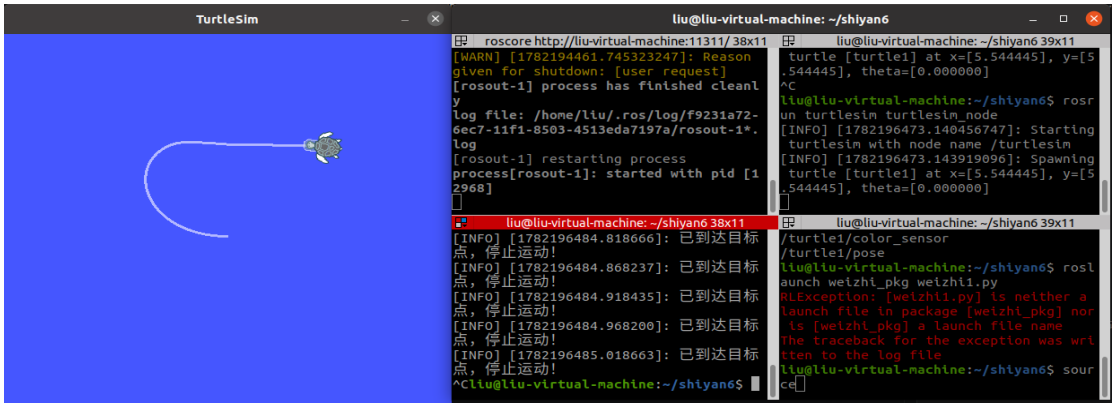
pub.publish() 将速度指令发送至话题，turtlesim 节点接收后驱动乌龟运动。

6.日志打印与周期延时

```
rospy.loginfo(f"目标位置: ({target_x:.2f}, {target_y:.2f}), "
              f"当前位置: ({current_x:.2f}, {current_y:.2f}), "
              f"距离误差: {distance:.2f}, "
              f"角度误差: {theta_error:.2f}")
rate.sleep()
```

◆rospy.loginfo() ROS 标准打印函数，终端带时间戳输出调试数据，用于观察误差收敛过程；

◆rate.sleep() 严格维持 20Hz 循环周期，无此行会全速死循环，CPU 爆满、控制周期混乱，PID 失效。



3.4 增量式 PID 乌龟点镇定

基于 ROS 与 Turtlesim 仿真，订阅乌龟位姿获取实际线速度，采用增量式 PID 稳速至 1.0m/s，固定角速度 0.5 使其转圈，每秒发控令并打印 PID 输出值。

3.4.1 增量式 PID 乌龟点镇定程序实现

```
#!/usr/bin/env python3
import rospy
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
```

#!/usr/bin/env python3：ROS Python 脚本标准声明，指定解释器；

◆rospy：ROS Python 核心库，实现节点、发布、订阅、定时；

◆Twist：速度消息类型，包含线速度 linear、角速度 angular，用来控制乌龟运动；

◆Pose：乌龟位姿消息，包含坐标、角度、实时线速度、角速度，用于反馈采集。

```
# 增量式 PID 参数
Kp=1.0
Ki=0.1
Kd=0.01
# 误差变量
e_current=0
e_last=0
e_prev=0
output=0
current_velocity = 0.0  # 乌龟当前实际线速度
```

三个 PID 核心系数

◆Kp 比例系数：快速缩小目标与实际值的偏差；

◆Ki 积分系数：消除静态误差；

◆Kd 微分系数：抑制震荡、提高系统稳定性；

◆增量式 PID 必须保存前两次误差：

◆e_current：本次误差；

◆e_last：上一周期误差；

◆e_prev：上两周期误差；

◆output：PID 累计输出控制量（最终给到乌龟 linear.x）；

◆current_velocity：全局变量，保存订阅获取的乌龟实时速度，作为反馈量。

```
def incrementalPID(setpoint,current):
    global e_current,e_last,e_prev,output
```

```
        e_current=setpoint-current
        delta_output=Kp*(e_current-
e_last)+Ki*e_current+Kd*(e_current-2*e_last+e_prev)
        output+=delta_output
        e_prev=e_last
        e_last=e_current
        return output
```

◆setpoint：设定值（本程序目标速度 1.0）；

◆current：反馈实际值（乌龟实时线速度）。

◆delta_output 增量输出，当前误差 $e(k)$=目标值 - 实际值，output 总输出=上一次输出 + 增量。

4.订阅回调函数

```
    def callback(pose_msg):
        global current_velocity
        current_velocity = pose_msg.linear_velocity
```

◆订阅/turtle1/pose 话题，每收到一条位姿消息自动执行回调；

◆pose_msg.linear_velocity：乌龟当前真实前进速度，作为闭环反馈；

◆存入全局变量，主循环随时读取做 PID 计算。

5. 程序入口主逻辑

```
    if __name__ == '__main__':
        rospy.init_node('turtle_velocity_controller')
        pub=rospy.Publisher('/turtle1/cmd_vel',Twist,queue_size=10)
        sub=rospy.Subscriber('/turtle1/pose',Pose,callback)
        rate=rospy.Rate(1)
        p=Twist()

        while not rospy.is_shutdown():
            # 目标速度设为 1，闭环调速
            p.linear.x=incrementalPID(1,current_velocity)
            p.angular.z=0.5  # 固定角速度，乌龟做圆周运动
            pub.publish(p)
            rospy.loginfo("linear velocity:%.2f",p.linear.x)
            rate.sleep()
```

◆rospy.init_node：创建 ROS 节点，名称 turtle_velocity_controller；

◆Publisher 发布器：

◆话题/turtle1/cmd_vel：乌龟速度控制指令话题；

◆消息类型 Twist，队列长度 10；

◆Subscriber 订阅器：订阅乌龟位姿话题，收到数据触发 callback 采集速度；

◆rospy.Rate(1)：控制循环频率 1Hz，1 秒执行一次 PID 计算与发布；

◆Twist()实例 p：存放本次要发送的速度指令；

◆主循环（ROS 标准死循环）：

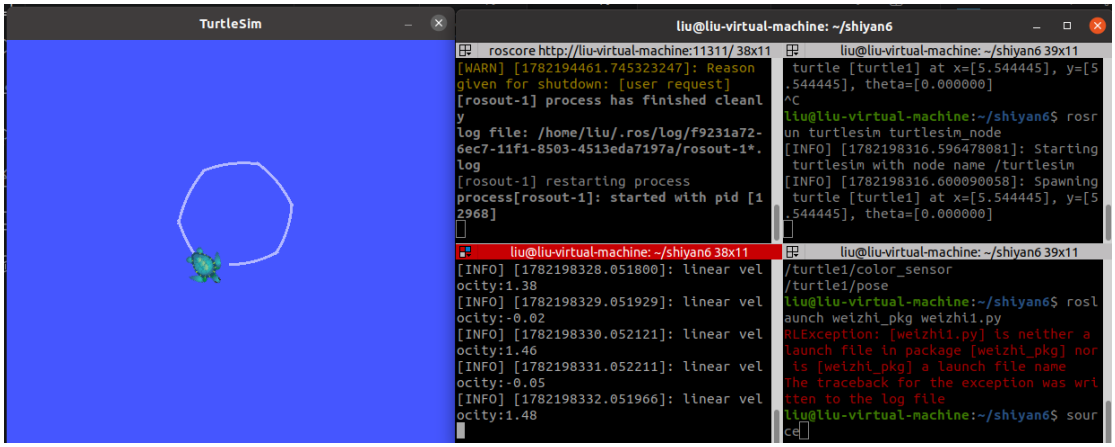
◆incrementalPID(1,current_velocity)：设定速度 1，传入实际反馈速度，得到 PID 输出，赋值给前进线速度；

◆angular.z=0.5：固定转向角速度，乌龟持续转圈；

◆pub.publish(p)：把速度指令发给小乌龟；

◆rospy.loginfo：终端打印 PID 输出的控制速度，方便观察调节效果；

◆rate.sleep()：保证 1Hz 运行周期。



五 实验任务

5.1 基础任务（必做）

1. 完成工作空间、功能包搭建，保证环境正常运行。

2. 理解公式与代码对应关系。

3. 位置式 PID 乌龟点镇定程序，观测运动现象，记录误差数据。

4. 运行增量式 PID 速度控制程序，观测速度变化规律。

5.2 提高任务

5.2.1 位置式 PID 分步参数整定与多目标点巡航

1. 基于位置式 PID 乌龟定点程序，完成参数整定 + 多点巡航双重改造：

2. 按照纯 P→PI→完整 PID 顺序整定参数，记录至少 2 组完整参数，观察静差、超调、震荡现象；

3. 在原有目标点 (8.0,8.0) 基础上新增 (2,9)、(9,2) 两组坐标，增加距离判断逻辑：乌龟与当前目标距离小于 0.1 则自动切换下一个点位；遍历全部点位后停止乌龟。

5.2.2 增量式 PID 段变速闭环调速实验

1. 原有固定目标线速度 1.0m/s 不变，新增分段切换逻辑：设置时间阈值，前 10s 目标速度 1.0m/s，10~20s 目标速度 0.4m/s，20s 后目标速度 1.5m/s，全程保持角速度 0.5 持续转圈；
2. 采集不同阶段 PID 输出速度、实时反馈线速度，记录三段速度切换瞬间的波动、调节时长；
3. 更换 1 组不同 Kp/Ki/Kd 参数，对比参数对变速切换时速度超调、恢复平稳速度所需时间的影响；

5.3 拓展任务（选做）

5.3.1 位置式 PID 增加积分限幅

原始位置式 PID 积分无限累加，远距离点位极易大幅超调冲点。修改 position_pid 函数，对积分和 error_sum 增加上下限钳位（如 ±5）；选用角落点位 (1,1) 对比加限幅 / 不加限幅两种工况的轨迹、超调量，总结积分限幅工程作用。

5.3.2 PID 面向对象实现位置式、增量式算法

摒弃全局变量写法，用 Python 类封装位置式、增量式 PID，将 Kp/Ki/Kd、误差缓存、积分和全部作为类成员变量；分别实例化角度控制器、距离控制器实现乌龟定点，对比全局变量与面向对象代码的可读性、复用性、多实例运行优缺点。

六 思考题

1. 简述比例、积分、微分环节各自作用，参数偏大 / 偏小会引发什么问题？
2. 位置式 PID 和增量式 PID 核心区别？结合本实验说明各自适用场景。
3. 什么是积分饱和？哪一种 PID 更容易出现该问题？如何缓解？
4. 本实验中角度做了归一化处理，去掉该段代码会出现什么现象？
5. 对比全局变量写法与面向对象写法，分析各自优缺点。